

OS_PROJECT03_10831_2021026108

컴퓨터소프트웨어학부 2021026108 김민제

notion을 이용하여 작성한 리포트입니다.

I. Copy-on-Write Fork

Design

Kalloc.c의 kmem 구조체에 ref_cnt라는 페이지 공유 추적 변수를 넣어서 관리하도록 했습니다.

uvmcopy에선 원래는 kalloc을 통해서 새롭게 할당해서 copy를 하지만 단순히 share를 하도록 수정했습니다.

usertrap에서는 pagefault가 났을 때 공유중이라면 새롭게 할당해서 쓰고 아니라면 그냥 그 페이지에 값을 씁니다.

copyout에선 공유하고 있다면 새롭게 만들어 그 곳에 값을 쓰고 아니라면 그냥 값을 써줍니다.

kalloc/kfree에선 위 동작이 올바르게 동작하도록 ref_cnt를 잘 추적하며 동작하도록 합니다.

Implementation

0. PTE_COW 추가

riscv.h에 #define PTE_COW (1L << 8) 를 추가했습니다.

1. kalloc.c

kmem 구조체에 uint ref_cnt[PHYSTOP / PGSIZE]; 를 추가하여 각 페이지마다 sharing 하는 프로세스의 수를 추적합니다.

- kalloc()에서는 ref_cnt값을 증가시켜줍니다.
- kfree()에서는 만약 더 이상 공유하는 프로세스가 없다면 정리하고 있다면 ref_cnt 값만 줄여줍니다.
- kgetrefcnt, kincrement, kdecrement를 추가하여 ref_cnt 관리를 쉽게하도록 했습니다.

2. trap.c

usertrap() 함수에서 r_scause() 가 15를 반환할 때 pagefault가 난것이므로 이 상황을 처리해주어야 합니다.

- pte 값을 확인해서 적절하지 않은 값이라면 에러를 일으킵니다.

```
if (pte == 0 || (*pte & PTE_V) == 0 || (*pte & PTE_U) == 0 ||
    ((*pte & PTE_W) == 0 && (*pte & PTE_COW) == 0))
    goto fail;
```

- page를 공유하지 않고 있다면 그냥 flag만 설정해줍니다.
- 만약 공유중이라면 새롭게 만들어주고 값을 설정하고 ref_cnt 값을 줄여줍니다.

```
char *mem = kalloc();
if (!mem)
    goto fail;
memmove(mem, (char *)pa, PGSIZE);
kdecrement((char *)pa);
*pte = PA2PTE(mem) | ((flags & ~PTE_COW) | PTE_W);
pa = (uint64)mem;
}
```

3. vm.c

- uvmcopy에서는 flag를 설정해주고 단순히 mapping만 해서 공유하도록 해줍니다.

```

for(i = 0; i < sz; i += PGSIZE){
    ...
    if(flags & PTE_W){
        flags &= ~PTE_W;
        flags |= PTE_COW;
        *pte = PA2PTE(pa) | flags;
    }
    flags = (flags & ~PTE_W) | PTE_COW;

    if(mappages(new, i, PGSIZE, pa, flags) != 0){
        return -1;
    }
    kincrement((char*)pa);
}

```

b. copyout에서는 값을 써줄 때 일어나는 것이므로 usertrap에서 Pagefault를 처리하는 부분과 동일하게 작성했습니다.

Results

제공받은 테스트 파일을 통해서 모든 명령들이 올바르게 동작하는지 확인했습니다.

makefile에 테스트 파일들을 등록해주고 테스트 파일들을 실행시켜 테스트를 진행했습니다.

```

init: starting sh
$ cowtest
simple: ok
simple: ok
three: ok
three: ok
three: ok
file: ok
ALL COW TESTS PASSED
$ echo 1
1
$ cowtest
simple: ok
simple: ok
three: ok
three: ok
three: ok
file: ok
ALL COW TESTS PASSED
$ █

```

그 결과 올바르게 동작하고 올바르게 종료가 되어 반복적으로 테스트 파일을 동작했을때도 이상없이 동작하는 것을 확인했습니다.

II. Large files

Design

file size를 키우기 위해서 여러 #defined 로 선언된 값들과 배열들의 크기를 늘려주었습니다.

처음 기본 상태는 1단계(indirect)로 접근을 하지만 2단계(hierarchy)로 접근해야해서 중간의 한 단계를 더 추가해주었습니다.

Implementation

0. param.h

FSSIZE를 2000에서 200000로 늘려주었습니다.

1. fs.h

a. NDIRECT를 12에서 11로 바꿔주었습니다.

- b. MAXFILE calculation을 #define MAXFILE (NDIRECT + NINDIRECT + NINDIRECT * NINDIRECT) 로 바꿔주었습니다.
- c. dinode 구조체의 addrs의 크기를 uint addrs[NDIRECT+2]; 로 수정해주었습니다.

2. file.h

- a. inode 구조체의 addrs의 크기를 uint addrs[NDIRECT+2]; 로 수정해주었습니다.

3. fs.c - bmap()

indirect는 여전히 접근하고 있고 추가로 double-indirect를 구현해주었습니다.

- a. bn -= NINDIRECT;로 indirect 탐색이 끝난 후의 bn을 초기화시켜주었습니다.
- b. bn이 원래 존재하던 범위를 넘어서 접근을 할 때는 double-indirect로 처리하도록 했습니다. Indirect로 접근하는거를 2번 반복하여 접근하는 느낌으로 수정했습니다. i, j를 2차원 배열 느낌으로 접근하도록 했습니다.

```
struct buf *db = bread(ip->dev, double_addr);
uint *indirect_blocks = (uint*)db->data;
uint i = bn / NINDIRECT;
uint j = bn % NINDIRECT;

uint indirect = indirect_blocks[i];
if (indirect == 0) {
    indirect = balloc(ip->dev);
    if (indirect == 0) {
        brelse(db);
        return 0;
    }
    indirect_blocks[i] = indirect;
    log_write(db);
}
brelse(db);

struct buf *ib = bread(ip->dev, indirect);
uint *data_blocks = (uint*)ib->data;

addr = data_blocks[j];
if (addr == 0) {
    addr = balloc(ip->dev);
    if (addr) {
        data_blocks[j] = addr;
        log_write(ib);
    }
}
brelse(ib);
return addr;
```

4. fs.c - itrunc()

xv6에서 파일의 데이터 블록들을 해제(**free**) 하는 역할을 하는 함수이므로 이때 접근을 2차원으로 하도록 했습니다. 원래 singly-indirect는 유지합니다.

- a. **Doubly-indirect block** 해제를 위해서 Doubly indirect block을 불러옵니다.
- b. 각 doubly의 indirect block 주소들을 순회하며 정리해줍니다.
- c. 마지막에 doubly indirect block 자체를 정리했습니다.

```

if (ip→addrs[NDIRECT + 1]) {
    bp = bread(ip→dev, ip→addrs[NDIRECT + 1]);
    a = (uint*)bp→data;
    for (i = 0; i < NINDIRECT; i++) {
        if (a[i]) {
            ibp = bread(ip→dev, a[i]);
            aa = (uint*)ibp→data;
            for (j = 0; j < NINDIRECT; j++) {
                if (aa[j])
                    bfree(ip→dev, aa[j]);
            }
            brelse(ibp);
            bfree(ip→dev, a[i]);
        }
    }
    brelse(bp);
    bfree(ip→dev, ip→addrs[NDIRECT + 1]);
    ip→addrs[NDIRECT + 1] = 0;
}

```

Results

제공받은 테스트 파일을 통해서 모든 명령들이 올바르게 동작하는지 확인했습니다.

makefile에 테스트 파일들을 등록해주고 테스트 파일들을 실행시켜 테스트를 진행했습니다.

대략 1시간 정도 bigfile 테스트를 돌았으며 성공적으로 예시 결과와 동일한 출력 결과를 얻었습니다. 이후의 동작들도 문제 없이 잘 동작했습니다.

```
$ bigfile  
.....  
.....  
.....  
.....  
wrote 65003 blocks  
bigfile done; ok
```

III. Symbolic links

Design

symlink 시스템 콜을 구현하고 open을 수정해서 T_SYMLINK file인 경우 10의 깊이 까지 recursive link following으로 찾아갈 수 있도록 했습니다.

Implementation

0. stat.h

```
#define T_SYMLINK 4
```

0. fctnl.h

```
#define O_NOFOLLOW 0x500
```

1. system call 등록

- syscall.h에 SYS_symlink 번호를 등록해주었습니다.
- syscall.c에서 extern과 배열에 mapping 시켜주었습니다.

2. user library 등록

- a. user.h에 symlink 시스템 콜을 등록해줍니다.

- b. `usys.pl`의 entry에 등록했습니다.

3. sysfile.c

- a. `create` 함수에서 동일한 파일을 생성할 때 `T_SYMLINK` type도 다를 수 있도록 했습니다.
- b. `sys_symlink` 함수를 추가했습니다.
 - a. str값을 인자로 `target`과 `path`를 받아옵니다.

```
if(argstr(0, target, MAXPATH) < 0 || argstr(1, path, MAXPATH) < 0){
    return -1;
}
```

- b. SYMLINK type의 파일을 생성합니다.

```
if((ip = create(path, T_SYMLINK, 0, 0)) == 0){
    end_op();
    return -1;
}
```

- c. `ip`에 선택한 `target`을 할당해주었습니다.

```
if(writei(ip, 0, (uint64)target, 0, strlen(target)) != strlen(target)){
    iunlockput(ip);
    end_op();
    return -1;
}
```

- c. `sys_open` 함수를 수정했습니다.

- a. `omode`가 `create`라면 생성을 하고 아니라면 해당 `path`의 파일을 찾습니다. 없다면 -1을 반환합니다.
- b. 파일을 발견했다면 `lock`을 걸어주고 `symlink`라면 `symlink file`의 형태를 따라서 처리해주었습니다.
- c. 이때 만약 `NOFOLLOW`로 설정이 되어있다면 `symlink`를 따라가지 않고 빠져나옵니다.
- d. `symlink` 순환을 방지하기 위해서 최대 10번만 깊이 탐색을 합니다.

```
int depth = 0;
while (ip->type == T_SYMLINK) {
    if (omode & O_NOFOLLOW) {
        break;
    }

    if (++depth > 10) {
        iunlockput(ip);
        end_op();
        return -1;
    }
    ...
}
```

- e. `symlink`가 가리키는 타겟 경로는 `readi()`를 사용하여 읽고, 이를 `namei()`로 다시 처리하여 해당 파일의 `inode`를 찾았습니다.

```
char target[MAXPATH];
int len = readi(ip, 0, (uint64)target, 0, MAXPATH);
iunlockput(ip);
if (len <= 0 || len >= MAXPATH) {
```

```
end_op();  
return -1;  
}  
target[len] = '\\0';  
  
ip = namei(target);
```

Results

제공받은 테스트 파일을 통해서 모든 명령들이 올바르게 동작하는지 확인했습니다.

makefile에 테스트 파일들을 등록해주고 테스트 파일들을 실행시켜 테스트를 진행했습니다.

```
init: starting sh  
$ symlinktest  
Start: test symlinks  
test symlinks: ok  
Start: test concurrent symlinks  
test concurrent symlinks: ok
```

Trouble Shooting

1. 첫번째 COW를 구현할 때 PA 기준으로 해야하는데 VA를 중간에 혼동하여 사용한 부분이 있어서 에러가 발생했었습니다.
2. 두번째 large file 구현에서 create를 수정하라 했는데 어떤 부분을 수정해야하는지 모르겠고 수정하지 않아도 잘 동작하여 특별히 수정하지 않았습니다.